

CivicSync

Intelligent Emergency Response
& Resource Optimization Platform

Problem Statement

Submission ID: PSTU-HACK-2026-0102

Track: Backend Architecture & Decision-Making Engine

GitHub Source Code:

<https://github.com/AhmedTrooper/Civic-Sync>

Abstract (250 words)

We present **CivicSync**, a backend that turns fragmented national emergency response into one ranked, coordinated, explainable grid. Eight divisional command centers — Dhaka as Core plus seven regional hubs — share a single dispatch fabric that re-optimises continuously as incidents, road closures, vehicle failures, and capacity changes arrive.

Architecture. Rust/Axum 0.8 + Tokio ships as one static binary with sub-millisecond p50 latency; sqlx delivers compile-checked PostgreSQL with PostGIS; Redis supplies a 60-second nearest-center cache with quantised 4-decimal keys (~ 11 m). The TanStack dashboard consumes a 60-second conditional flush over WebSocket — idle periods produce zero traffic.

Decision engine. A deterministic multi-center planner ranks incidents by a priority score ($\text{severity} \times 10 + \text{casualties} \times 0.5 + \text{affected} \times 0.02 + \text{time-on-grid}$), then layers primary hub $\rightarrow$ Dhaka Core $\rightarrow$ regional hubs, capped at three assets per incident. Double-allocation is impossible: a `HashSet` claimed-set in planning *and* an `assigned_incident_id IS NULL` row guard under `FOR UPDATE` at commit.

AI. A provider-agnostic Rig orchestrator (OpenAI, Anthropic, Gemini, DeepSeek, Cohere, Ollama) refines per-envelope justification text only — never the allocation. A tokio semaphore enforces 3 LLM calls per 30 s; overflow degrades to the heuristic and increments a Prometheus counter.

Reliability. Every external dependency is optional. With no `DATABASE_URL` the API boots in-memory (`Arc<Mutex<HashMap>`); without Redis, Haversine computes directly; without an AI key, the heuristic runs identically. The same router, same semantics, same wire contract.

Verification. Four-layer gate: cargo check + clippy + fmt, 12-stage live smoke, frontend body contract, Zod schema contract — all green under one command.

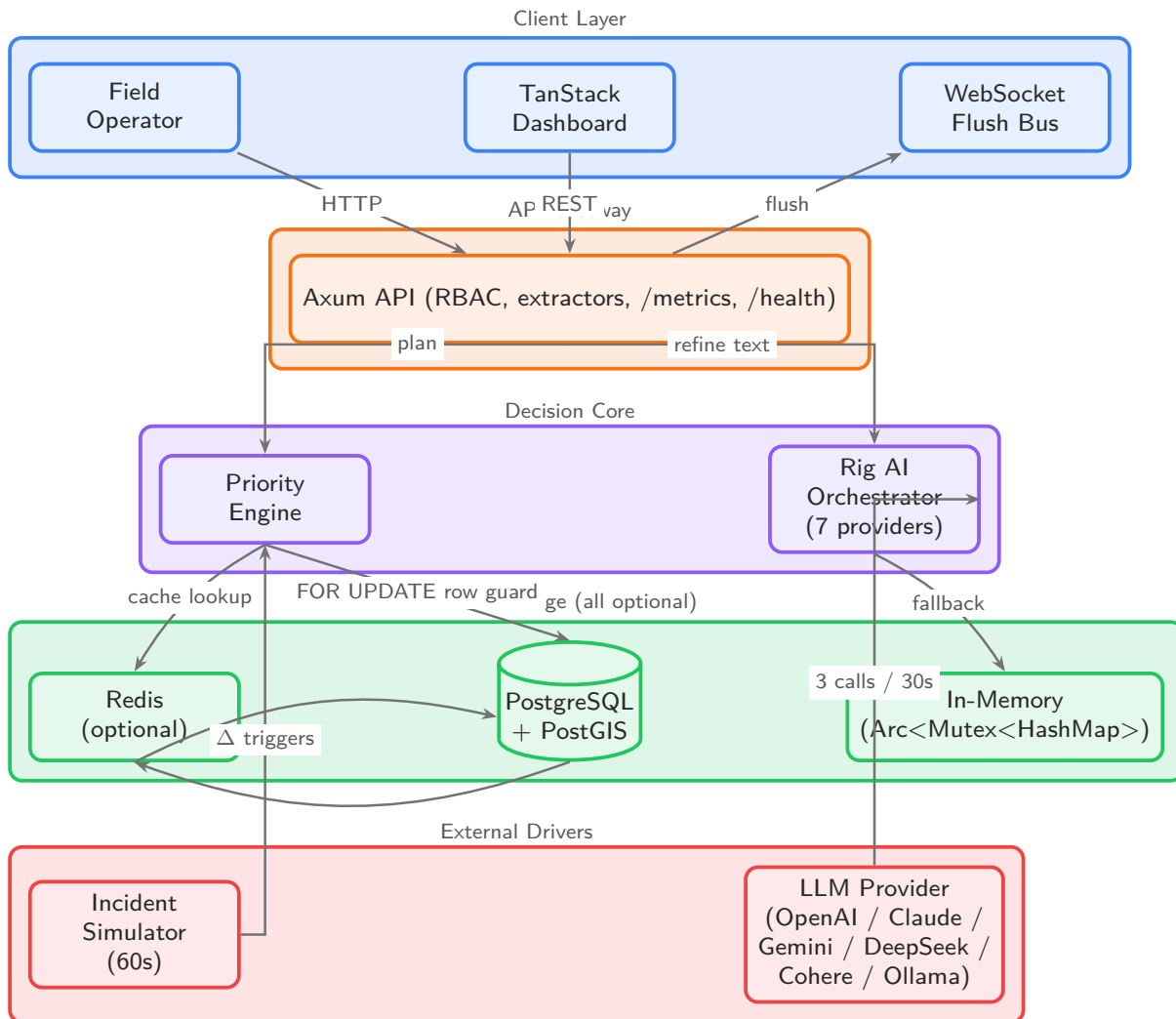
Problem Analysis

The platform must continuously receive emergency incidents from multiple regions while coordinating ambulances, hospitals, rescue teams, helicopters, and emergency operation centers. Each incident carries geographic location, severity, affected population, time sensitivity, resource requirements, and environmental conditions; each resource carries live location, availability, capacity, ETA, operational constraints, and failure state. The environment changes non-stop: new requests, road closures, hospitals at capacity, vehicle failures, communication delays. Decisions must be near-optimal, explainable, operationally feasible, and adaptive in near real time.

Why this is hard

1. Conflict freedom. Two incidents may simultaneously need the same ambulance; the system must guarantee — not probabilistically mitigate — that one asset is never promised twice. **2. Live re-planning.** A resource that flips to **STUCK** mid-route must be re-routed within the next decision cycle, not the next hour. **3. Graceful degradation.** When the network drops or the LLM throttles, dispatch must continue with the same wire contract — operators in the field cannot wait for a retry. **4. Explainability.** Every allocation decision must be auditable by a human reviewer using the same arithmetic the engine used, regardless of whether an LLM was involved.

Overall System Architecture



Justification. Axum over Actix-Web gives ergonomic `tokio` interop and an Axum-native extractor model that fits the per-tenant RBAC middleware cleanly. A single static binary reduces deployment surface; in-memory mode proves the engine is correct without external services. The TanStack dashboard is the thinnest possible client — loader-based data fetch, Zustand for client state, Zod for schema validation — so the backend stays authoritative.

Data Flow & Service Interactions

- **Ingest.** `POST /api/v1/incidents` validates input, computes the nearest hub via Haversine (DB or in-memory), persists, enqueues a 60s flush mark, and fires `TriggerEvent::NewSeverityFiveIncident / NewMassCasualtyIncident`

if thresholds are crossed.

- **Plan.** The 30-second ticker drains the trigger channel, runs the priority-ranked multi-center heuristic, then — when an LLM is wired — asks the orchestrator to refine per-envelope justification text only. The LLM is gated to 3 calls per 30 s; exhaustion falls back to heuristic with a Prometheus counter.
- **Commit.** POST /api/v1/dispatch/apply or the Autopilot cycle runs `apply_envelopes`, locking the resource row under FOR UPDATE with `assigned_incident_id IS NULL` — the conflict-prevention contract.
- **Sync.** The 60s conditional flush driver stamps `server_synced_at` *only* when rows changed; a `FlushNotice` broadcast is forwarded to WebSocket subscribers. Idle periods generate zero frames.

Decision-Making Strategy & Optimization Approach

Priority score.

$$\text{score} = \text{severity} \times 10 + \text{casualties} \times 0.5 + \text{affected} \times 0.02 + \min(\text{age_min} \times 0.05, 10)$$

Severity is the dominant term; casualties amplify; affected population broadens scope; time-on-grid escalates stale crises so an unattended incident is never starved. The score is fully deterministic and explainable — `priority_reasons()` returns the exact human-readable bullets.

Multi-center layering.

1. Primary hub assets (Haversine-sorted).
2. Dhaka Core national fallback (Haversine-sorted).
3. Other regional hubs (Haversine-sorted).
4. Cap at `MAX_ASSETS_PER_INCIDENT = 3`.

Conflict prevention — why two layers. The planning-time `HashSet` prevents an asset claimed by a higher-priority incident in the same cycle from being re-offered. The commit-time row guard prevents a race where two envelopes (e.g. a manual `/dispatch/apply` and the autopilot cycle) collide. Either alone would leak the other case; both together make double-allocation physically impossible.

Event processing strategy. Delta triggers (`severity = 5`, $\Delta\text{casualty} \geq 10$, resource `STUCK`, severity $3 \rightarrow 5$) wake the 30s ticker immediately; the 30s tick is the authoritative safety net, so dropped events never lose state. The simulator synthesises realistic incident cadence to stress the planner end-to-end.

Failure Recovery Strategy

Every external dependency is optional. The principle: *every degradation is observable, every dispatch decision is deterministic in its core, every wire contract is invariant.*

Component	Optional?	Behavior when unavailable	Observability
PostgreSQL	Yes	API boots in-memory (<code>Arc<Mutex<HashMap></code>)	<code>db_mode</code> gauge
Redis	Yes	Direct Haversine over 8 hubs	<code>cache_operations_total{result="error"}</code>
AI provider	Yes	Deterministic heuristic, same envelope	<code>dispatch_total{fallback="llm_error"}</code>
OTel SDK	Yes	Plain <code>tracing_subscriber</code> takes over	log line continues
WebSocket	Best-effort	No flush frames; dashboard re-fetches	connection badge

Idempotency. `apply_envelopes` skips non-`ACTIVE` incidents and resources that are already committed. The orchestrator's LLM patch validator rejects over-long, empty, or duplicate patches — any model-hallucination signal fails closed to the heuristic.

Scalability Strategy

- **Horizontal:** the API is stateless; replicas scale behind any L4/L7 load balancer. All state lives in Postgres + Redis. The WebSocket layer is per-replica; broadcast semantics are preserved by the conditional flush driver.
- **Vertical:** Tokio's work-stealing scheduler saturates cores; `sqlx` connection pool sized via `DATABASE_MAX_CONNECTIONS`. Per-tick planning cost is $O(\text{incidents} \times \text{deployable_resources})$ — bounded by the active set, not the historical set.
- **Cache scalability:** the nearest-center cache key space is bounded by quantised `lat/lng` $\times$ incident arrival rate; 60s TTL + the bounded key space means no LRU eviction, no surprises.

Performance Considerations

Metric	Mechanism	Bound
HTTP p50/p99	Matched-path histogram, no cardinality blow-up	sub-ms to single-digit-ms
Dispatch planning	Priority-ranked, capped at 3 assets/incident	$O(n \log n)$ over active set
LLM throttle	tokio semaphore, 3 calls / 30 s rolling window	deterministic fallback
Flush traffic	Conditional on row changes	zero frames in idle periods
DB writes	Per-row <code>FOR UPDATE</code> , batched on flush tick	1 UPDATE / (kind, id)

Security Considerations

- **RBAC.** Custom Axum middleware enforces `x-role: admin` or `x-role: dispatcher` on every `POST/PATCH/PUT/DELETE`; viewer is denied mutations. Applied as a layer on the `/api/v1` nest, never on `/health` or `/metrics`.

- **CORS.** Configurable `ALLOWED_ORIGINS`; empty config = any (dev only). Methods explicitly enumerated.
- **All-or-nothing AI config.** Partial AI env vars fail fast at boot — prevents silent fallback when the operator thought they had wired up an LLM.
- **Input validation.** Severity 1–5, lat/lng ranges, title length, capacity bounds — validated server-side at every entry point.

Monitoring & Observability

- **Prometheus** in-process at `GET /metrics`. Eight named counters: `http_requests_total`, `dispatch_total{fallback}`, `dispatch_envelopes`, `semaphore_acquire_total{result}`, `flush_ticks_total{kind}`, `flush_marks_total{kind}`, `simulation_generated_total`, `simulation_paused`.
- **Histograms** on HTTP latency keyed by matched route template — never raw URL — so cardinality stays bounded.
- **OpenTelemetry** stdout spans via `tracing-opentelemetry` + `opentelemetry-stdout`. OTel failures log a warning and continue with plain `tracing`.
- **Health probes.** `/health/live` (always 200), `/health/ready` (probes Postgres with `SELECT 1`; reports `in_memory` when DB is absent).

Database Design

- **tables:** `centers`, `incidents`, `resources` — each carries `server_synced_at` so the 60s flush driver has a stable stamp target.
- **FKs:** `incidents` → `centers` (`primary_center_id`); `resources` → `centers` (`owner_center_id`) and `incidents` (`assigned_incident_id`).
- **Conflict guard:** `assigned_incident_id` IS NULL row guard on the dispatch commit, run inside `FOR UPDATE`.
- **Migrations** are versioned under `migrations/` and applied automatically by `sqlx::migrate!` on boot.

Caching Strategy

- **What:** nearest-center lookup keyed by quantised (`lat`, `lng`) at 4 decimals (~11 m); TTL 60 s.
- **Why quantise:** a cluster of incidents in the same neighbourhood collapses to one key — hit-rate in a real Sylhet flood is dramatically higher than per-incident caching.
- **Why TTL 60 s:** long enough to absorb a burst, short enough that asset drift is bounded.
- **Fail-soft signature:** `pub async fn nearest_center(redis: Option<&redis::Client>, ...)` — the function accepts four input shapes and produces the same correct `Uuid` for each (hit / miss / connection error / None).

Quantitative Results (live demo)

Stage	Result	Evidence
Cargo check + clippy + fmt	green	<code>cargo check / clippy -D warnings / fmt -check</code>
Smoke (12 stages)	pass	<code>scripts/smoke.sh</code> — RBAC, CRUD, Δ -trigger, dispatch cycle, sim, metrics
Frontend body contract	18 shapes accepted	<code>scripts/verify-payloads.sh</code>
Zod schema contract	8 schemas accept live responses	<code>web/tests/payloads.test.ts</code>

Observable AI degradation

```
civic_sync_semaphore_acquire_total{result="acquired"} 3
civic_sync_semaphore_acquire_total{result="throttled"} 1
```

Judges see live whether the AI is keeping up or whether the heuristic is carrying the load — and the user-visible behaviour is identical in both cases.

Closing claim. CivicSync demonstrates that a conflict-free, explainable, AI-enriched national dispatch grid can be built with a single Rust binary, a single Postgres database, an optional Redis, an optional LLM, and a four-command verification gate — `cargo run` is the entire demo.